MyCodeSchool

Angular — fișă rapidă

User Management fără server · toate noțiunile din exerciții · Sebastian

1. Comenzi

`npm install`	instalează pachetele (o dată, după clonare)
`npm start`	pornește aplicația → `http://localhost:4200` · oprire: `Ctrl+C`
`npx ng build`	verifică dacă proiectul compilează

2. Componenta — două fișiere

```
import { Component, signal, computed } from '@angular/core';
interface User { id: number; firstName: string; age: number; }

@Component({
  selector: 'app-user-list',            ← tag-ul
  templateUrl: './user-list.html',      ← fișierul .html
})
export class UserList {
  users = signal<User[]>([...]);        ← starea
  canSave = computed<boolean>(...);     ← calculat
  remove(id: number): void { ... }      ← acțiuni
}
```

```
<h1>Utilizatori ({{ users().length }})</h1>

@for (user of users(); track user.id) {
  <tr> ... </tr>
} @empty {
  <tr><td colspan="3">Niciun utilizator</td></tr>
}
```

`.html` vede doar ce e în clasă (de aceea `Number()` nu merge acolo).

3. TypeScript

`interface User { id: number; ... }`	forma unui obiect	`User[]`	listă de User
`User | null`	un User sau nimic	`nextId: number = 6`	câmp cu tip
`remove(id: number): void`	metodă cu tip	`this.users()`	în clasă se scrie cu `this.`
`cond ? a : b`	una din două	`` `${a} ${b}` ``	text compus
`a !== b · && · !x`	diferit · și · nu	`this.nextId++`	crește cu 1

4. Template

`{{ users().length }}`	afișează	`(click)="remove(user.id)"`	la click
`<input #first [value]="firstName()" (input)="firstName.set(first.value)">`	input legat de semnal	`(input)="age.set(years.valueAsNumber)"`	citește un număr
`@if (cond) { } @else { }`	condiție	`@if (toDelete(); as user) { }`	dacă nu e `null`, îl numești
`@for (u of users(); track u.id) { } @empty { }`	listă; `@empty` = goală	`[disabled]="!canSave()"`	buton inactiv
`{{ user.firstName[0] }}`	prima literă	`@`	scrii @ ca text

5. Semnale

`x = signal<number>(0)`	valoare care redesenează ecranul	`x()`	citește (și în `.html`)
`x.set(5)`	valoare nouă	`x.update((v) => v + 1)`	din cea veche
`canSave = computed<boolean>(() => this.firstName() !== '')`		se recalculează singur	
`this.users().push(u)`		NU: ecranul nu află → `update` cu listă nouă	

6. Liste și texte

`this.users.update((list) => [...list, user])`	adaugă la final	`list.length`	câte sunt
`this.users.update((list) => list.filter((u) => u.id !== id))`	șterge	`text.toLowerCase()`	litere mici
`this.users.update((list) => list.map((u) => u.id === id ? nou : u))`	înlocuiește	`text.includes("pop")`	conține?
`for (const u of this.users()) { sum = sum + u.age; }`	adună	`Math.round(x * 10) / 10`	o zecimală

7. Ce folosești la fiecare exercițiu

1	`interface User`	6	`update` + `filter`, `(click)`
2	`import`, `signal<User[]>`	7	`#box` + `[value]` + `(input)`, `valueAsNumber`, `[...list, user]`, `nextId`
3	`{{ users().length }}`	8	`editedId`, `map` pentru înlocuire, `? :`
4	`@for` + `track`, `@empty`	9	`computed`, `[disabled]`
5	`@if` / `@else`	Bonus	`toLowerCase` + `includes`, `for…of`, `User | null` + `@if (…; as user)`